

Wide but Shallow

Technical-Interview Problems Are Four Ideas in Many Costumes

and company-specific preparation is mostly a myth

Conor Garvey Gelvin
gelvin@outlook.com

Data snapshot: June 2026 · Draft

Abstract

Interview-prep advice rests on two claims: that different companies test different skills, and that you must memorize dozens of separate problem “patterns.” We test both against how often each problem appears among seven major technology firms’ most-asked lists (769 distinct problems), and both mostly fail. The roughly twenty “patterns” you are told to learn are really **four underlying ideas**: a single left-to-right pass (a *fold*), divide-and-solve recursion (dynamic programming), spreading information across a graph until it settles, and binary search. A fixed rule, written in advance, places **about three-quarters of all problems** (75%) in one of these four; the rest form a genuine “tail.” The single pass is the most common, and many tricks taught as unrelated—prefix sums, the XOR trick, a hash “seen set,” reversing a list, the sliding window—are the *same* pass; they differ only in what running value they carry. To check those labels are real rather than merely plausible, we re-prove them with the Lean proof assistant: on a *pre-registered* random sample, **54 of 100 problems**—three-quarters of those the rule assigns to a scheme—carry a machine-checked proof that the assigned idea really solves *that* problem, and not one is refuted. Separately, the seven firms ask almost the same mix: on a 0-to-1 scale of how differently they ask (bias-corrected Cramér’s V , where 0 is identical), they score just 0.091, above the 0.065 chance level but far below “moderate”—six of the seven are statistically indistinguishable, and only Uber stands out (it leans on graph problems). We describe which problems are *commonly asked*, not how hard they are or whether company-specific drilling pays off. In short: interview problems are *wide on the surface but shallow underneath*, and essentially one shared syllabus covers nearly every company. All derived data and proofs are released; the raw scrape is withheld per the platform’s terms.

1 Two myths

Two beliefs dominate interview preparation. The first is *company specificity*: that Google, Meta, Amazon and their peers test different skills, so you should prepare differently for each. The second is *pattern proliferation*: that the field is a long list of loosely related tricks—sliding window, monotonic stack, union-find, prefix sums, and so on—each learned separately, as popular pattern catalogues present them [1]. Both are widely repeated and, as far as we can tell, never tested.

We test them, and then ask what underlying *structure* explains the answer. The short version: the “fifty patterns” you are told to memorize are a handful of ideas in different disguises, and the seven companies ask nearly the same things. To keep our labels honest we do not hand-sort the problems and stop there—we use a proof assistant to check, problem by problem, that the idea we assign really does solve the problem (and, for several showcase examples, that the standard solution is fully correct).

2 Data and method

For seven firms (Amazon, Apple, Bloomberg, Google, Meta, Microsoft, Uber) we scraped a public interview-prep platform for how often each problem appears among that firm’s *most-asked* problems (the platform exposes a top-200 list per firm and recency window, so rarely-asked problems are truncated; we note this where it matters). Across firms this covers 769 distinct problems, broken down by recency. The platform tags each one with a *surface family* (“sliding window,” “monotonic stack,” “dynamic programming,” and so on), which we consolidate into about twenty broad families.

A fixed rule, written down in advance (Appendix A), maps each family to one of our four underlying *ideas* (formally, *recursion schemes*)—a coarser label based on the *shape* of the standard solution rather than its textbook name (Table 1)—or to a leftover “tail” for problems that fit no single idea. We ask two separate questions and keep them apart. *Coverage*—what fraction of problems the four ideas explain—is simply how often the rule lands a problem in a scheme rather than the tail; it is a property of the rule, stable across samples (Section 3). The proof assistant (Section 4) then does something different and stronger: on a pre-registered sample it *verifies*, problem by problem, that the rule’s labels are real. Separately, the *company* comparison—do firms ask different mixes?—weights each idea by how often it is asked (Section 3); it groups by the four ideas, with the finer twenty-family view alongside as a robustness check, and all difference measures are bias-corrected [3] and reported with uncertainty.

3 Result: four ideas, not fifty

Wide surface. The surface taxonomy really is spread out: no one or two families dominate (it takes six of the ~ 20 to cover even half of asked problems; Gini 0.30 over distinct problems). By the usual measure, interview preparation *looks* like a long, flat list. This is the breadth the prep industry sells.

Shallow root. But that breadth is mostly repackaging. Sorting the same problems by the *shape* of their standard solution, a fixed rule (Appendix A) places about three-quarters of them in just four ideas—75% of all 769, and 68–82% of any random sample of 100—with the single left-to-right pass the most common. Tricks memorized as unrelated—prefix sums, the XOR trick, a hash “seen set,” reversing a list, matching parentheses with a stack, the sliding window—are the *same* pass; they differ only in what running value they carry. The variety is in the bookkeeping, not in the computation. The remaining quarter—bespoke data structures, simulation, geometry, ad-hoc number theory—is a genuine tail we do not force into a scheme. Section 4 then machine-checks that these labels are real, not just plausible.

One shared syllabus. The same data dissolve the company myth. We score how differently the firms ask on a standard 0-to-1 scale (bias-corrected Cramér’s V [2, 3], where 0 means identical and the usual cutoffs call 0.1 “small” and 0.3 “moderate”). Across the four ideas the firms score just $V = 0.091$ (95% confidence interval [0.06, 0.12])—small, with the whole interval well below “moderate,” so the difference is genuinely bounded, not merely undetected. We report the four-idea grouping because its score stands clear of the small difference that random chance alone would produce in a sample this size (0.091 against a chance level of 0.065); at the finer twenty-family level the score falls *below* that chance level (0.064 against 0.116)—indistinguishable from identical—so the finer view, if anything, makes the firms look more alike, not less. Comparing firms two at a

Recursion scheme	textbook families it subsumes	assigned	certified	Lean witness
streaming fold	two-pointers, sliding-window, monotonic-stack, hashing, prefix-sum, linked-list, bit-XOR, string-scan	29	23	IsFold
recursive decomposition (DP)	dynamic programming, backtracking, tree	25	18	catamorphism
graph relaxation	BFS/DFS, Dijkstra, Bellman–Ford, topo-sort, union–find	11	6	least fixpoint
bisection	binary search (monotone predicate)	7	7	monotone threshold
in a scheme		72	54	
not a scheme	design, simulation, heaps, number theory, string matching, ...	28	—	—

Table 1: The roughly twenty surface families collapse onto four recursion schemes. On a pre-registered random sample of 100 distinct problems (the seed fixed before the draw; Section 4): “assigned” is how many the fixed rule (Appendix A) places in each scheme—72, in line with the rule’s 75% rate over the full corpus; “certified” is how many additionally carry a Lean proof that the idea solves *that* problem, not just some problem in the scheme. 54 of the 72 in-scheme problems (three-quarters) are certified, and none is refuted; the binomial uncertainty on that verification rate is $[0.44, 0.63]$ over the whole sample, but coverage itself is the rule rate, not this number. The “Lean witness” is the formal test each scheme is checked against (a *catamorphism* is a fold over a tree; a *least fixpoint* is where graph relaxation settles). The 18 uncounted in-scheme problems are *unproven, not disproven*: each needs a multi-step proof beyond one self-contained file (full DP optimality, minimum-spanning-tree cost, parsing, ...). Among the 54, strength varies: full correctness for the flagship folds of Section 4, a one-directional property (e.g. “everything it reports is valid”) otherwise.

time, the only real differences all involve **Uber**, which asks more graph problems and fewer single-pass ones (its largest gap, versus Google, is still only “small”; Figure 1); **the other six firms are statistically indistinguishable**. One caveat: we compare the *mix of problem types*, not how hard the problems are or whether drilling for a specific company helps—our data cannot measure those. Within that scope, there is essentially one interview syllabus, and preparing differently per company is, for most firms, wasted effort.

4 Machine-checking the classification

The coverage number is only as trustworthy as the rule’s labels. The usual check—a second human rater—only tells you whether two people read a label the same way. We do something stronger: we check the labels with a *proof assistant*, a tool that mechanically verifies mathematical claims. Working in Lean 4 [4] with its library Mathlib [5], we write each of the four ideas as a precise test on programs (for example, “this function is a single left-to-right pass”), and for each sampled problem we prove two things. First, **the idea fits** (`c1s`): the standard solution really does have the

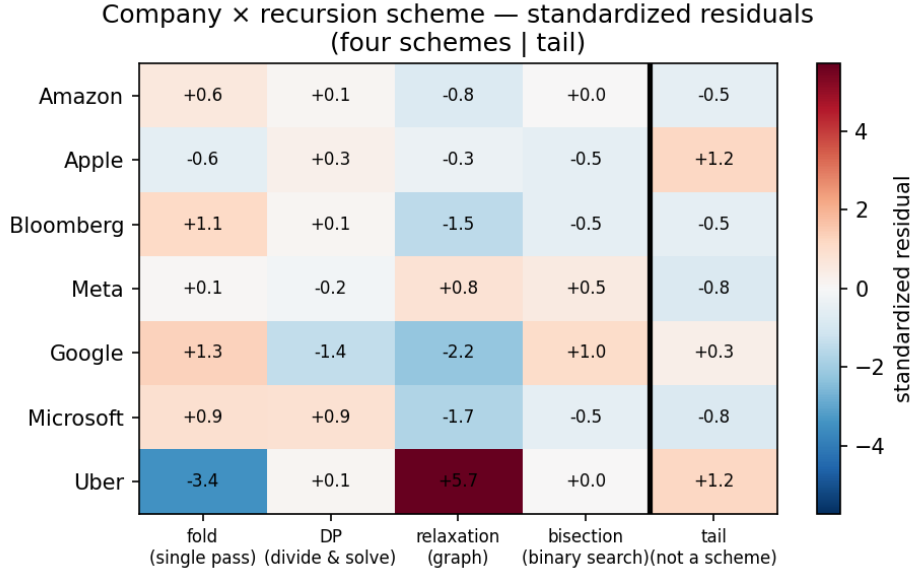


Figure 1: How much each firm over- or under-asks each idea, relative to the shared average (standardized residuals; red = more than expected, blue = less). Only Uber stands out—it asks far more graph problems (+5.7) and far fewer single-pass ones (−3.4)—while the other six rows are muted (all within about ± 2). This is the picture behind the small overall difference ($V = 0.091$): one shared syllabus, with Uber the lone exception.

shape of its assigned idea. Second, **the solution is sound** (*corr*): that solution provably satisfies a property of the *specific* problem—its actual grid, array, or condition—not a generic statement every problem in the family would satisfy. That last requirement is what matters: a proof stated over an abstract relation certifies the *category*, not the *problem*. A problem counts only when both proofs pass with no gaps (Lean flags any unfinished proof; there are none), use only Lean’s standard axioms, and meet the problem-specific bar; the gate that enforces this ships as a script, so the count is mechanical, not a matter of judgement.

A pre-registered sample. The count depends on which problems are sampled, because not every label is equally easy to formalize. An earlier draft drew its sample from a seed we had, in effect, chosen well: we had proved that one sample to near-completion, and a sample worked to completion measures how hard we worked, not how often the labels hold—checked afterward, that seed was the best-scoring draw out of 2000. We discarded it, fixed a new seed *before drawing* (recorded in the released `PREREGISTRATION.md`), and report whatever it yields. On that sample, **54 of 100 problems** carry a problem-specific certificate: 54 of the 72 the rule assigns to a scheme (three-quarters), with the 28 tail problems correctly left unassigned. Crucially, **no label is refuted**—every in-scheme problem we could formalize confirmed its assigned idea.

What is left uncounted. The other 18 in-scheme problems are unproven, not disproven. Each needs a multi-step proof beyond a single self-contained file—full dynamic-programming optimality (e.g. Best Time to Buy and Sell Stock IV), minimum-spanning-tree cost, expression parsing, cycle detection, computational geometry—which we leave honestly out rather than count something weaker. We likewise exclude proofs stated over an abstract relation even when they compile: a few graph problems whose only formal content is “the reachable set is the least fixpoint” certify reachability in general, not the specific problem.

What this does and does not show. The two proofs cover an example of the idea and a working solution, but we do not separately prove those two are the *same* program. So a certificate says “this problem is solvable by idea S , with a verified-correct solution,” not “the platform’s exact accepted code is literally a single pass.” For a few textbook cases we close even that gap: Kadane’s algorithm for Maximum Subarray is literally one pass carrying a running pair, and we prove it returns exactly the largest subarray sum; Single Number is the XOR pass proven to return the lone unpaired value. These are the strong form the general procedure approximates for the rest. We do not re-prove the deep theory behind the ideas—that shortest paths and dynamic programming are formally well-founded is already established [6, 7, 8, 9], which we cite rather than redo.

5 Related work

Two literatures bracket this paper. On the *empirical* side, interview preparation is organized by pattern catalogues [1] that enumerate surface families; we are not aware of prior work that tests whether those families are generatively distinct, or whether firms differ in the mix they ask. On the *structural* side, the observation that wide families of programs are instances of a few recursion schemes is the Bird–Meertens tradition of program calculation [10] and its formal-methods descendants: associative aggregation over a sliding window is a verified fold [11], dynamic programming is verified memoized recursion [9], and shortest paths are a least fixpoint over a semiring [6, 7, 8]; verified-algorithms textbooks now develop these schemes end to end [12, 13]. The four-scheme decomposition itself is therefore not new—it is the classical Bird–Meertens repertoire. Our contribution is to connect that structural lens to the *empirical* distribution of interview problems (the company-comparison test), and to machine-check the scheme assignment per problem rather than assert it.

6 Threats to validity

One platform. The frequencies come from a single prep platform and may reflect its own collection process as much as what employers truly ask; we describe *commonly-asked* problems, not any firm’s private bar. **Top-200 cap.** Each firm’s frequencies are its *most-asked* top-200 list per recency window, not the full distribution, so the rare tail is truncated; the company comparison is over the heads of the distributions, where the data are densest. **Borrowed labels.** The family-to-idea rule is ours, but the surface-family tags it starts from are the platform’s—single-source and un-audited—so the “wide surface” count inherits whatever splits that taxonomy chose. **The model is a choice.** The proof checks that a problem *can* be solved by an idea; it cannot certify that “which idea it is” is the right way to describe what an interview tests. That a problem “is a single pass” is our modeling choice—the proof confirms the model holds, not that it is the only sensible one. **Idea and solution proven separately.** As in Section 4, we prove the idea fits and, separately, that a solution is sound; proving the two are the same program is done only for the showcase examples and is otherwise future work. **Coverage is a rule rate; the proof count is a check.** The 75% headline is how often a fixed rule lands a problem in a scheme—stable across samples (68–82%) and needing no proofs. It errs low in one direction: the rule sends borderline cases (some “greedy” and “simulation” problems that are really single passes or binary searches) to the tail rather than into a scheme, so 75% is, if anything, a floor. The 54/100 is a different quantity—a per-problem *check* that the labels hold—and is lower than 75% only because some labels need multi-file proofs we did not attempt, never because a label failed. **Window dependence.** The company difference is not constant across recency windows: the bias-corrected V ranges from below its noise floor in

the short windows up to 0.125 in the six-month window (still “small”), so “one shared syllabus” is a statement about the pooled data, not every slice. **Two separate questions.** Coverage counts each distinct problem once; the company comparison weights by how often a problem is asked. Both matter to a candidate, and we keep them apart. **Shallow is not easy.** “Same idea” does not mean “same difficulty”—two single-pass problems can differ hugely in how hard the running value is to find. We claim shallow *structure*, not that the problems are easy.

7 Conclusion

Interview problems are wide on the surface but shallow underneath. A fixed rule places about three-quarters of them in four ideas; on a pre-registered random sample, three-quarters of the in-scheme problems carry a machine-checked proof that the assigned idea really solves them, and none is refuted (the single left-to-right pass is the most common). And the seven firms ask essentially one shared syllabus. For a candidate, the takeaway is blunt: learn the four ideas deeply rather than fifty patterns shallowly, and prepare for “technical interviews,” not for any one company.

Reproducibility and AI-use disclosure

All derived statistics, the family-to-idea rule (released as a standalone table, fixed in advance so anyone can check the categories were not tuned to fit the sample), the pre-registration of the sampling seed (`PREREGISTRATION.md`, committed before the sample was drawn or scored), the certified list (from which the 54/100 re-derives), and the complete Lean development are released. The problem-specific gate that decides which compiled certificates count is a released, mechanical script, so the count is reproducible rather than a matter of our judgement. The raw per-company frequency tables are withheld under the platform’s terms, so the headline numbers reproduce from the released derived data even though the scrape itself is not redistributed. The proofs build with `lake build` with no gaps; the “standard axioms” claim is checkable, not merely asserted, by running `#print axioms` on the main theorems. This paper was written by one human author with substantial AI assistance for drafting, statistical code, and the formalization; all claims, numbers, and proofs were checked by the author, and the machine-checked results are independently verifiable by re-running the proofs. We say so plainly: the paper’s central claims stand or fall on the released data and the proofs, not on how it was written.

References

- [1] S. Prashad. LeetCode Patterns. <https://seanprashad.com/leetcode-patterns/>, 2020.
- [2] H. Cramér. *Mathematical Methods of Statistics*. Princeton University Press, 1946.
- [3] W. Bergsma. A bias-correction for Cramér’s V and Tschuprow’s T . *Journal of the Korean Statistical Society*, 42(3):323–328, 2013.
- [4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction (CADE-28)*, 2021.
- [5] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP)*, 367–381, 2020.

- [6] B. Nordhoff and P. Lammich. Dijkstra’s shortest path algorithm. *Archive of Formal Proofs*, 2012.
- [7] A. Armstrong, G. Struth, and T. Weber. Kleene algebra. *Archive of Formal Proofs*, 2013.
- [8] D. J. Lehmann. Algebraic structures for transitive closure. *Theoretical Computer Science*, 4(1):59–76, 1977.
- [9] S. Wimmer, S. Hu, and T. Nipkow. Monadification, memoization and dynamic programming. *Archive of Formal Proofs*, 2018.
- [10] R. Bird and O. de Moor. *Algebra of Programming*. Prentice Hall, 1997.
- [11] L. Heimes, D. Traytel, and J. Schneider. Formalization of an algorithm for greedily computing associative aggregations on sliding windows. *Archive of Formal Proofs*, 2020.
- [12] T. Nipkow, J. Blanchette, M. Eberl, A. Gómez-Londoño, P. Lammich, C. Sternagel, S. Wimmer, and B. Zhan. *Functional Algorithms, Verified!* 2021. <https://functional-algorithms-verified.org>.
- [13] T. Nipkow, M. Eberl, and M. P. L. Haslbeck. Verified textbook algorithms: A biased survey. In *Automated Technology for Verification and Analysis (ATVA)*, 2020.

A Family-to-scheme assignment rule

Each surface family is mapped to the scheme matching the *shape of its canonical accepted solution*: a single left-to-right pass carrying one accumulator $\rightarrow$ *streaming fold*; a recurrence that decomposes a structure into sub-results $\rightarrow$ *recursive decomposition (DP)*; iterative improvement toward a fixed point over a graph $\rightarrow$ *graph relaxation*; a monotone predicate located by halving $\rightarrow$ *bisection*; anything else (bespoke data structures, simulation, ad-hoc number theory, string matching) $\rightarrow$ *tail*. The rule is applied to the family, not tuned per problem; the per-problem Lean checks (Section 4) then test whether the assigned scheme actually solves each sampled problem.